

EECS 348 Assignment 3: Comparing Two AI-Generated Programs

Francis Tryban · EECS 348 · October 1, 2026

For this assignment, I gave two GenAIs the same C++ problem, tested what they made, and then improved the one I thought was stronger. The generated code below is included exactly as the models wrote it. I used Claude Opus 5.5 to help make the larger test script and turn my notes into an earlier draft. OpenAI Codex helped me edit this analysis. My comparison is based on the programs and the saved test results in this project.

2a-2b. The two GenAIs

I used **Claude Haiku 4.5** through Claude Code and **GPT-6 Luna** through the Codex, each in their respective desktop app. Both ran on my Mac, each in its own identical folder. Both models on their Light effort setting. I used larger models for Assignment 2, so this time I wanted to see what the smaller models would do with the same kind of problem.

2c. The prompt

I put the same program instructions in both `Claude/PROMPT.md` and `Codex/PROMPT.md`. Both `AGENT.md` files were empty for this first run. I gave each model the same run instruction: read `AGENT.md` and `PROMPT.md`, write the C++ program, and save it as `iterations/00-raw.cpp`. Neither model was given the rubric or knew they were in a competition

Here is the full program prompt both models received:

```
Write C++ code using objects, not functions, to implement the following r

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A
  priority queue is a queue where emails can shift towards the front of t
  queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then us
  that MaxHeap to handle all your email prioritizing for the CEO.
- You must create objects from scratch. Do not include pre-existing heap
```

modules.

Test File format:

Command	Description
`EMAIL <sender category>,<subject line>,<date>`	The emails in the CEO
`NEXT`	Next email for the CEO to read. Display the information on the
`READ`	CEO has read email and dealt with it
`COUNT`	Display current count of unread emails

The grader will test your code with a different test file. The test file begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be format as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

Sample Test File:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

Your program should display this:

```
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
```

Subject: Important
Date: 12-20-2024
There are 4 emails to read.

Input clarification

The program's expected input is a text file containing one command per line such as:

...

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL Boss,Important,12-20-2024

COUNT

NEXT

READ

...

The program must accept that file in either of two ways:

- If a filename is given as the first command-line argument, read the commands from that file: `./program testfile.txt`
- If no argument is given, read the commands from standard input:
`./program < testfile.txt`

All output goes to standard output. The program exits with status 0 when it completes normally.

2d. The programs they generated

Claude Haiku 4.5 – Claude/iterations/00-raw.cpp

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <sstream>
using namespace std;
```

```

bool dateGreater(string date1, string date2) {
    int mm1 = stoi(date1.substr(0, 2));
    int dd1 = stoi(date1.substr(3, 2));
    int yyyy1 = stoi(date1.substr(6, 4));

    int mm2 = stoi(date2.substr(0, 2));
    int dd2 = stoi(date2.substr(3, 2));
    int yyyy2 = stoi(date2.substr(6, 4));

    if (yyyy1 != yyyy2) return yyyy1 > yyyy2;
    if (mm1 != mm2) return mm1 > mm2;
    return dd1 > dd2;
}

class Email {
public:
    string sender;
    string subject;
    string date;
    int priority;

    Email() : sender(""), subject(""), date(""), priority(0) {}
    Email(string s, string sub, string d, int p)
        : sender(s), subject(sub), date(d), priority(p) {}

    bool operator>(const Email& other) const {
        if (priority != other.priority) {
            return priority > other.priority;
        }
        return dateGreater(date, other.date);
    }
};

class MaxHeap {
private:
    vector<Email> heap;

    int parent(int i) {
        return (i - 1) / 2;
    }

    int leftChild(int i) {
        return 2 * i + 1;
    }
}

```

```

int rightChild(int i) {
    return 2 * i + 2;
}

void bubbleUp(int i) {
    while (i > 0 && heap[i] > heap[parent(i)]) {
        swap(heap[i], heap[parent(i)]);
        i = parent(i);
    }
}

void bubbleDown(int i) {
    int largest = i;
    int left = leftChild(i);
    int right = rightChild(i);

    if (left < (int)heap.size() && heap[left] > heap[largest]) {
        largest = left;
    }

    if (right < (int)heap.size() && heap[right] > heap[largest]) {
        largest = right;
    }

    if (largest != i) {
        swap(heap[i], heap[largest]);
        bubbleDown(largest);
    }
}

public:
    void push(Email email) {
        heap.push_back(email);
        bubbleUp(heap.size() - 1);
    }

    Email pop() {
        Email max = heap[0];
        heap[0] = heap[heap.size() - 1];
        heap.pop_back();
        if (heap.size() > 0) {
            bubbleDown(0);
        }
    }

```

```

        return max;
    }

    Email peek() const {
        return heap[0];
    }

    bool empty() const {
        return heap.size() == 0;
    }

    int size() const {
        return heap.size();
    }
};

int getPriority(string category) {
    if (category == "Boss") return 5;
    if (category == "Subordinate") return 4;
    if (category == "Peer") return 3;
    if (category == "ImportantPerson") return 2;
    if (category == "OtherPerson") return 1;
    return 0;
}

string trim(string s) {
    size_t start = s.find_first_not_of(" \t\n\r");
    size_t end = s.find_last_not_of(" \t\n\r");
    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

int main(int argc, char* argv[]) {
    MaxHeap emailQueue;

    istream* input = &cin;
    ifstream file;

    if (argc > 1) {
        file.open(argv[1]);
        input = &file;
    }

    string line;

```

```

while (getline(*input, line)) {
    if (line.empty()) continue;

    stringstream ss(line);
    string command;
    ss >> command;

    if (command == "EMAIL") {
        string sender, subject, date;
        getline(ss, sender, ',');
        getline(ss, subject, ',');
        getline(ss, date);

        sender = trim(sender);
        subject = trim(subject);
        date = trim(date);

        int priority = getPriority(sender);
        Email email(sender, subject, date, priority);
        emailQueue.push(email);
    }
    else if (command == "COUNT") {
        cout << "There are " << emailQueue.size() << " emails to read\n";
    }
    else if (command == "NEXT") {
        if (!emailQueue.empty()) {
            Email current = emailQueue.peek();
            cout << "Next email:" << endl;
            cout << "Sender: " << current.sender << endl;
            cout << "Subject: " << current.subject << endl;
            cout << "Date: " << current.date << endl;
        }
    }
    else if (command == "READ") {
        if (!emailQueue.empty()) {
            emailQueue.pop();
        }
    }
}

if (argc > 1) {
    file.close();
}

```

```
    return 0;
}
```

GPT-6 Luna – Codex/iterations/00-raw.cpp

```
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

class Email {
public:
    std::string sender;
    std::string subject;
    std::string date;
    int senderPriority;
    int dateKey;
    unsigned long long sequence;

    Email(const std::string& senderValue, const std::string& subjectValue,
          const std::string& dateValue, unsigned long long sequenceValue)
        : sender(senderValue), subject(subjectValue), date(dateValue),
          senderPriority(0), dateKey(0), sequence(sequenceValue) {
        if (sender == "Boss") senderPriority = 5;
        else if (sender == "Subordinate") senderPriority = 4;
        else if (sender == "Peer") senderPriority = 3;
        else if (sender == "ImportantPerson") senderPriority = 2;
        else senderPriority = 1;

        // MM-DD-YYYY is converted to YYYYMMDD so integer order matches c
        if (date.size() == 10) {
            dateKey = (date[6] - '0') * 10000000 + (date[7] - '0') * 1000000 +
                      (date[8] - '0') * 100000 + (date[9] - '0') * 10000 +
                      (date[0] - '0') * 1000 + (date[1] - '0') * 100 +
                      (date[3] - '0') * 10 + (date[4] - '0');
        }
    }

    bool outranks(const Email& other) const {
        if (senderPriority != other.senderPriority)
            return senderPriority > other.senderPriority;
    }
};
```



```

        if (dateKey != other.dateKey) return dateKey > other.dateKey;
        return sequence < other.sequence;
    }
};

class MaxHeap {
private:
    std::vector<Email> items;

    void swapItems(std::size_t first, std::size_t second) {
        Email temp = items[first];
        items[first] = items[second];
        items[second] = temp;
    }

    void siftUp(std::size_t index) {
        while (index > 0) {
            std::size_t parent = (index - 1) / 2;
            if (!items[index].outranks(items[parent])) break;
            swapItems(index, parent);
            index = parent;
        }
    }

    void siftDown(std::size_t index) {
        while (true) {
            std::size_t left = index * 2 + 1;
            std::size_t right = index * 2 + 2;
            std::size_t best = index;
            if (left < items.size() && items[left].outranks(items[best]))
                best = left;
            if (right < items.size() && items[right].outranks(items[best]))
                best = right;
            if (best == index) break;
            swapItems(index, best);
            index = best;
        }
    }

public:
    bool empty() const { return items.empty(); }
    std::size_t size() const { return items.size(); }

    void insert(const Email& email) {
        items.push_back(email);
        siftUp(items.size() - 1);
    }
};

```

```

}

const Email* top() const {
    return items.empty() ? nullptr : &items[0];
}

void removeTop() {
    if (items.empty()) return;
    if (items.size() == 1) {
        items.pop_back();
        return;
    }
    items[0] = items.back();
    items.pop_back();
    siftDown(0);
}

};

class EmailQueueProgram {
private:
    MaxHeap inbox;
    unsigned long long nextSequence = 0;

    void handleEmail(const std::string& line) {
        std::string fields = line.substr(6);
        std::size_t firstComma = fields.find(',');
        std::size_t secondComma = fields.find(',', firstComma == std::string::npos);
        if (firstComma == std::string::npos || secondComma == std::string::npos) return;

        std::string sender = fields.substr(0, firstComma);
        std::string subject = fields.substr(firstComma + 1, secondComma - firstComma - 1);
        std::string date = fields.substr(secondComma + 1);
        inbox.insert(Email(sender, subject, date, nextSequence++));
    }

    void handleLine(const std::string& line) {
        if (line.compare(0, 6, "EMAIL ") == 0) {
            handleEmail(line);
        } else if (line == "COUNT") {
            std::cout << "There are " << inbox.size() << " emails to read\n";
        } else if (line == "NEXT") {
            const Email* email = inbox.top();
            if (email == nullptr) return;
            std::cout << "Next email:\n";
        }
    }
};

```

```

        << "Sender: " << email->sender << '\n'
        << "Subject: " << email->subject << '\n'
        << "Date: " << email->date << '\n';
    } else if (line == "READ") {
        inbox.removeTop();
    }
}

public:
    void run(std::istream& input) {
        std::string line;
        while (std::getline(input, line)) {
            if (!line.empty() && line.back() == '\r') line.pop_back();
            handleLine(line);
        }
    }
};

int main(int argc, char* argv[]) {
    std::ifstream file;
    std::istream* input = &std::cin;
    if (argc > 1) {
        file.open(argv[1]);
        if (!file) return 0;
        input = &file;
    }

    EmailQueueProgram program;
    program.run(*input);
    return 0;
}

```

2e. My comparison

I read both programs before testing them. Claude's was easier for me to follow at first, but Codex's was organized more like the assignment asked: its email data, heap, and command handling are all inside classes. Claude put some of that work in separate functions and handled every command in `main`.

Correctness

Both programs compiled and matched the assignment's sample output exactly. I tried the sample with the filename on the command line and with the file piped into the program. Both ways worked. I also

checked the cases the assignment specifically calls out: an empty inbox, two `NEXT` commands in a row, and two `READ` commands in a row. Both handled those without crashing. In a larger test of 500 emails, both returned every email in the expected priority order.

There were still differences worth noticing:

Test	Claude	Codex
Same sender and same date, three emails	Changed the order of the tied emails	Kept their original order
Extra spaces around a command or email field	Handled them	Sometimes ignored a command or sorted a date incorrectly
Missing or nonnumeric date	Could crash when comparing emails	Did not crash, but could silently skip or misread the email
Missing input file	Finished without a useful error	Finished without a useful error

The assignment does not say what order tied emails should have, so I would not call Claude's result wrong. I do prefer Codex's predictable answer. The assignment also promises properly formatted commands. I tested bad input anyway because a program should give me some clue when something went wrong. Neither first draft did that well. The detailed results are saved in `evidence/run00-bench-local.txt`.

Execution time

Both use a heap built on a list. Adding or removing an email takes more steps as the inbox grows, but the work grows slowly: about $O(\log n)$ for one add or remove, and $O(n \log n)$ for a full file of emails. Looking at the code alone, they seem similar. The timing test showed a real difference.

I ran the same workload through each program: add the emails, then use `NEXT` and `READ` to work through them. These are user CPU times from the plain build, using the best of three runs on the same Linux machine.

Number of emails	Claude	Codex
5,000	0.021 s	0.005 s
50,000	0.266 s	0.059 s
500,000	3.252 s	0.777 s

At 500,000 emails, Codex was about **4.2 times faster**. Claude works out the date again whenever it compares two emails from the same sender group. Codex works out a number for the date once when the email is added. In a separate trial, changing only Claude's date handling dropped its time from about 3.35 to 1.10 seconds. That convinced me the repeated date work caused most of the gap.

Space complexity

Both keep their emails in a growing list, so both use **$O(n)$** space: twice as many emails means roughly twice as much storage. Codex also saves each email's arrival order so it can settle ties. That makes one Codex email 112 bytes compared with 104 bytes for Claude's.

With 100,000 emails in the inbox, the measured peak memory was about **19.0 MB for Claude** and **20.3 MB for Codex**. Claude used about 8% less. I think the extra memory is reasonable because it gives Codex a consistent order when two emails have the same sender and date.

Maintainability

Neither raw program included the required opening comment with my name, program description, inputs, outputs, and sources. Claude had **no comments** in its code; Codex had **one**. Both needed work here before I could submit either one.

Claude's short names and using `namespace std` made it easier for me to read line by line. Codex was easier to work on as a whole. Its `main` is only 13 lines, while Claude's is 58 lines and handles all four commands there. Claude also has three helper functions outside its classes; Codex puts that work inside objects. I checked how many decisions are packed into one function: the highest score was 11 for Claude and 7 for Codex. Both are under Dr. Johnson's limit of 15, but Codex's work is spread out more clearly.

2f. Which program I chose

I chose **Codex's program** as the starting point. Both got the required sample and the normal email order right. Codex fit the "objects, not functions" requirement better, was much faster in the large test, and gave tied emails a predictable order. Its weak spots were mostly things I could add or clean up: comments, input checks, and useful error messages. Claude's speed problem and its long `main` would take more restructuring. I accepted Codex's small memory cost because the tie order was useful and the measured difference was only about 1.3 MB with 100,000 emails.

2g. How I improved it

I saved the improved program as `email_priority.cpp`. I put writing and behavior rules in `Codex/AGENT.md`, then asked GPT-6 Luna to revise its first program. I reviewed that revision, and made a couple of changes naimly taking codex's signature off of EVERY comment.

- **Correctness:** The program now removes extra spaces around commands and email fields. An `EMAIL` line with a missing field, an unknown sender, or a date in the wrong format is skipped with an explanation. `NEXT` on an empty inbox says `No emails to read`. A missing input file or an extra command-line argument gives an error and an unsuccessful exit. The sample output still matches exactly, and the final version still returned all 500 test emails in the right order.
- **Execution time:** I kept Codex's faster approach of working out the date once per email. The first revision accidentally became slower because it used `std::endl` on every output line, which makes the program send output immediately each time. I had those lines changed back to `'\n'`. On the same 500,000-email test, the first Codex version took 0.788 seconds, the revision took 0.938 seconds, and the final version took 0.869 seconds. The final version is a little slower than the first because it checks the input, but it is still far faster than Claude's version.
- **Space complexity:** The final program still uses $O(n)$ space. An email still takes 112 bytes, and the measured peak memory stayed the same. I kept the arrival number because removing it would lose the consistent tie order.
- **Maintainability:** I added the required opening comment, including my name and the AI sources. I added comments explaining the major sections and individual lines, with a note on each major section saying where its code came from. Each command now has its own method, so the command handling is easier to find and change. Only `main` remains outside a class.

I ran the test script again after the last edits. The final version still matched the sample exactly through both input methods, passed the 500-email order check, and produced the same results as the revision on the tested cases. Those results are saved in `evidence/runfinal-bench-local.txt`. These tests were run on Linux, but not on the course's Cycle server. The final executable file was compiled on linux to be put in the git repo.